Tide retrospective: *Quartic and sextic as k-th specialisations (A1 follow-up refactor)*

Daniel Murfet

18 May 2026

Abstract

Follow-up refactor to A1 (generic even-monomial 1D template): replaces 13 explicit-proof theorems across the quartic and sextic files with thin specialisations of A1's generic theorems. Net deletion 162 lines (quartic 386 → 287, sextic 403 → 340). Single session, no roadblocks, single-attempt clean build (8298 jobs), zero `sorry` / `axiom` / `native_decide`. Marked as a refactor rather than a tide: no deliberation needed (A1's deliberation handled the question of whether this refactor was worth doing), and the work is mechanical (one bridge lemma + a pattern of `convert ... using 3` per theorem).

1 Setting

A1 landed earlier today: the generic-k template for $L_k(x) = x^{2k}/(2k)!$, with nine theorems. GPT's A1 consult recommended deferring the quartic / sextic refactor to a separate follow-up tide, on the grounds that mixing abstraction with downstream rewrites makes review noisier. This Tide is that follow-up.

The value: makes A1's abstraction visible. Before, the quartic and sextic files each carried ∼25-line proofs of theorems that A1 now subsumes; after, those theorems are 3–5 line specialisations. The integrability lemmas (`quartic_integrable_pow*` and `sextic_integrable_pow*`) are kept since A1 deferred them.

2 The thing formalised

Two bridge lemmas:

- `quarticPotential_eq_kthPotential : quarticPotential = kthPotential 2`
- `sexticPotential_eq_kthPotential : sexticPotential = kthPotential 3`

Each proves by `funext; simp; norm_num` (definitional up to Nat reduction of $(2 \cdot 2)! = 24$ and $(2 \cdot 3)! = 720$).

Then 13 thin specialisations replacing the explicit proofs:

- Quartic ($k = 2$, 7 theorems): the half-line integral, the even and odd moments, the partition function (+ positivity), and the even / odd expected values.

- Sextic ($k = 3$, 6 theorems): same minus the half-line integral, which carries an arbitrary-m signature that's strictly more general than A1's '2j'-keyed version, so it stays as-is.

Each specialisation follows the pattern

```
rw [_Potential_eq_kthPotential]   -- when partitionFunction or
                                   -- gibbsExpectation is involved
have h := kth_* (k := 2 or 3) (by norm_num) ... ht
convert h using 3
```

where `convert ... using 3` handles the Nat-cast plumbing between the specific form (24, 4 in the exponents) and the generic form $((2 \cdot 2)!, ((2 \cdot 2 : \mathbb{N}) : \mathbb{R}))$.

3 Proof strategy

Mechanical. The bridge lemma carries the work: once `quarticPotential` = `kthPotential` 2 is in scope, the partitionFunction and gibbsExpectation versions specialise directly. The moment integrals don't need the bridge because they already use raw $\exp(-tx^4/24)$ form, which is definitionally equal to $\exp(-tx^{2 \cdot 2}/((2 \cdot 2)! : \mathbb{R}))$ after Nat reduction — `convert` handles this transparently.

4 Roadblocks and resolutions

None. Each of the 13 theorems went green on first attempt with the same `convert ... using 3` pattern.

A small mid-test diagnostic: the bridge proof initially used `funext; simp` alone, which left $x^4/24 = x^4/\uparrow$(Nat.factorial 4) unsolved. Adding `norm_num` closes the residual Nat-cast.

5 What was Mathlib, what was new

From Mathlib. `norm_num`, `convert`, `funext`, `simp`.

From the seabed. A1's nine theorems (consumed verbatim).

New here. Two bridge lemmas (5 lines each); 13 specialisation proofs (3–5 lines each) replacing the original ∼25-line proofs.

6 Lessons

The `convert ... using 3` pattern handles Nat-reduction noise transparently. The specialised theorems' statements are in the "human" form (24, 4); the generic theorem's RHS is in the "Nat-cast" form $((2 \cdot 2)!, ((2 \cdot 2 : \mathbb{N}) : \mathbb{R}))$. The two are equal by Nat computation but not syntactically. `convert ... using 3` walks the goal three levels deep and emits the cast-equality side goals, all of which close by definitional Nat reduction. Worth recording as the standard specialisation idiom.

Bridge lemmas should be definitional whenever possible. The bridge identities are true by definitional unfold plus Nat reduction; each bridge proof is one line. The alternative — redefining the specific potentials directly as the generic one — would break unfolds elsewhere in downstream files, so the bridge-as-lemma approach is strictly less disruptive.

7 Follow-ups

- *Integrability tide.* Still pending. A1 deferred `kth_integrable_pow*` and the quartic / sextic versions remain self-contained. Once the integrability tide lands, the quartic / sextic files can shrink another ∼100 lines.
- *Refactor downstream specific lemmas.* The $n = 1$ expected-value, $n = 0$ linear case, and the affine covariance are kept verbatim — they already consume the even/odd theorems internally, so no further gain.[1] Skip.
- *Sextic half-line integral.* The sextic's half-line integral takes an arbitrary $m : \mathbb{N}$ (not just $2 \cdot n$). A1's analogue is keyed on $2 \cdot j$ and is therefore weaker. The sextic version was kept verbatim. A genuinely-arbitrary-m generic version in A1 would supersede it; deferred until needed.

Acknowledgements

GPT-5.5 Pro's A1 consult set the structural decision to do the refactor as a separate tide, which is what made this clean. No fresh GPT consult was needed.

[1] Specifically `quartic_expected_value_sq`, `quartic_expected_value_lin`, and `quartic_cov_affine`.